

Ваш эксперимент четко доказывает наличие **электрического конфликта на шине I2C**. Это классическая проблема при параллельном подключении китайских модулей.

Вот в чем физическая причина происходящего:

1. Проблема «лишней» подтяжки

Шина I2C требует резисторов подтяжки (pull-up) к 3.3В.

- В **WT32-ETH01** встроенной подтяжки на GPIO 32/33 нет.
- В модуле **DS3231** (обычно это плата ZS-042) уже впаяны резисторы **4.7 кОм**.
- В модуле **BMP280** (фиолетовая платка) также впаяны резисторы **10 кОм**.
- Ваши **4.7 кОм** были третьими в цепи.

Когда вы всё это соединяете, суммарное сопротивление падает слишком низко (закон Ома для параллельных цепей). Емкость шины становится критической, и ESP32 просто не может «прожать» линию до логического нуля, либо делает это слишком медленно.

2. Конфликт уровней и «диодная» защита

Популярные модули **DS3231 (ZS-042)** имеют встроенную цепь подзарядки аккумулятора (диод + резистор), которая часто подключена к VCC. Если вы питаете часы от 5В, а датчик от 3.3В, то через резисторы подтяжки часов на шину I2C просачивается 5В. Это «ослепляет» BMP280 и может мешать часам инициализироваться.

Как сделать, чтобы работало всё вместе (Проверенный рецепт):

1. **Питание:** Убедитесь, что **оба** модуля питаются строго от **3.3В**. Не подавайте 5В на DS3231.
2. **Резисторы:**
 - Уберите свои внешние 4.7 кОм совсем.
 - Оставьте резисторы только на **одном** модуле (лучше на часах).
 - Если на модуле BMP280 есть крошечные резисторы (маркировка 103 или 472), попробуйте их аккуратно сдуть феном или просто доверьтесь подтяжке от DS3231.
3. **Длина проводов:** Для I2C на ESP32 провода должны быть не длиннее **15-20 см**. Если они длиннее, шина начинает ловить наводки от работающего Ethernet.
4. **Скорость шины:** В коде в `setup()` после `Wire.begin(32, 33)` добавьте принудительное снижение скорости:

```
Wire.setClock(100000); // Снижаем до 100 кГц (Standard Mode)
```

Это делает шину гораздо более помехоустойчивой.

Почему BMP280 выдавал 786.0 hPa?

Это происходит, когда инициализация прошла, но в момент чтения данных шина «дребезжит». Датчик возвращает некорректный байт, который при расчете превращается в это число.

Предлагаю: Подключите оба устройства, запитайте оба от 3.3В, уберите свои внешние резисторы и добавьте `Wire.setClock(1000000)`.

Появились ли оба устройства в логе после снижения скорости до 100 кГц?

вопрос, как заставить работать одновременно эти два устройства?

Чтобы заставить **DS3231** и **BMP280** работать стабильно на одной шине I2C в связке с **WT32-ETH01**, нужно устранить аппаратную «перегрузку» линий и программные конфликты.

Вот пошаговая инструкция:

1. Аппаратная часть (Критично)

Модули DS3231 (особенно синие платы ZS-042) — главные «вредители» на шине.

- **Удалите лишние резисторы:** Уберите ваши внешние 4.7кОм. На модуле DS3231 уже стоят 4.7кОм, а на BMP280 — 10кОм. Вместе их достаточно.
- **Единое питание 3.3В:** Подайте на VCC обоих модулей строго 3.3В. Если подать на DS3231 5В, его подтяжки «подтянут» шину к 5В, что заблокирует BMP280 или выжжет порты ESP32.
- **Проверка модуля DS3231 (ZS-042):** На этих платах есть цепь зарядки (диод + резистор 201). Если вы используете обычную батарейку CR2032 (а не аккумулятор LIR2032), **выпаяйте этот диод или резистор**. Эта цепь часто вносит помехи в питание, которые «вешают» шину I2C.

2. Программная часть (Настройка шины)

ESP32 по умолчанию пытается работать на 400 кГц, что при наличии двух устройств и наводок от Ethernet приводит к сбоям. Нужно «замедлить» шину.

В `setup()` сделайте так:

```
void setup() {  
  Serial.begin(115200);
```

```

// 1. Инициализация шины на стандартной скорости
Wire.begin(32, 33);
Wire.setClock(100000); // Снижаем до 100 кГц для стабильности
delay(100);

// 2. Сначала запускаем RTC
if (rtc.begin()) {
    LOG("RTC", "DS3231 found");
} else {
    LOG("RTC", "DS3231 NOT found");
}

// 3. Небольшая пауза между инициализациями
delay(50);

// 4. Запускаем барометр (обязательно указываем адрес)
if (bmp.begin(0x76)) {
    LOG("BMP280", "Sensor found at 0x76");
} else {
    LOG("BMP280", "BMP280 NOT found");
}
}

```

3. Разделение опроса во времени

Главная проблема в `loop()` — попытка прочитать оба датчика мгновенно один за другим.

В блоке `readSensors()` сделайте разнос:

```

void readSensors() {
    static uint32_t lastRead = 0;
    if (millis() - lastRead < 2000) return; // Опрос раз в 2 секунды
    lastRead = millis();

    // Читаем время
    DateTime now = rtc.now();

    delay(10); // Микропауза для успокоения шины

    // Читаем давление
    float p = bmp.readPressure();
    if (p > 80000 && p < 120000) { // Фильтр "мусора" 786 hPa
        pressure = p / 100.0F;
    }
}

```

```
}  
}
```

Резюме:

1. Питание обоих модулей — **3.3В**.
2. Внешние резисторы — **убрать**.
3. Скорость шины в коде — **100 кГц**.
4. Добавить **delay** между обращениями к устройствам.

Попробуйте снизить скорость до 100 кГц (`Wire.setClock(100000)`) — это обычно решает 90% проблем с «невидимыми» часами.

оба устройства подключены к шине питания 3.3В, стоит блокировочная емкость 820мкф расстояние между модулями примерно 1 см. на часах стоит подтяжка 4.7к, на датчике давления подтяжка 10к.

Учитывая, что модули находятся в 1 см друг от друга и имеют суммарную подтяжку около **3.2 кОм** ($\$1/R = 1/4.7 + 1/10\$$), электрически шина перегружена, но не критично. Однако на таком сверхкоротком расстоянии и при наличии мощного Ethernet-контроллера рядом, возникают паразитные наводки.

Чтобы они наконец заработали вместе, сделайте следующее:

1. Программный «разнос» (Самое важное)

ESP32 пытается опросить устройства слишком быстро. Шина I2C не успевает «отдохнуть» после ответа одного модуля перед запросом к другому.

В `setup()` примените строго такой порядок:

```
Wire.begin(32, 33);  
Wire.setClock(100000); // Обязательно 100 кГц  
delay(200);           // Пауза для стабилизации питания после включения  
  
if (rtc.begin()) {  
    Serial.println("[RTC] OK");  
}  
delay(100);           // Критическая пауза! Даем RTC "отпустить" шину
```

```
if (bmp.begin(0x76)) { // Проверьте, точно ли 0x76, а не 0x77
  Serial.println("[BMP280] ОК");
}
```

2. Проблема модуля DS3231 (ZS-042)

Если у вас стандартный китайский модуль с батарейкой, проверьте **цепь заряда**. На нем стоит диод и резистор (201), которые пытаются заряжать батарейку от 3.3В. Это создает микро-шумы на шине питания, которые "вешают" чувствительный BMP280.

- **Решение:** Выпаяйте (или скovyрните) маленький диод на плате часов. Это никак не повредит часам, но уберет шум в цепи 3.3В.

3. Резисторы подтяжки

Поскольку расстояние всего 1 см, емкость линии минимальна. Суммарная подтяжка 3.2 кОм для ESP32 на такой длине может быть избыточной.

- **Попробуйте:** Оставьте резисторы **только на часах** (4.7к), а на BMP280 удалите (или наоборот). Для 1 см одного комплекта резисторов более чем достаточно.

4. Проверка адреса BMP280

Иногда библиотека по умолчанию ищет 0x77, а модуль настроен на 0x76. Если вы вызываете `bmp.begin()` без адреса, он может «стучаться» не туда и вешать шину.

- Всегда пишите явно: `bmp.begin(0x76)`.

Что делать сейчас:

1. Добавьте `Wire.setClock(100000)` и `delay(100)` между инициализациями.
2. Проверьте адрес датчика через I2C Scanner (запустите его, когда подключены оба устройства).

I2C Scanner видит оба адреса (0x68 и 0x76/0x77) одновременно при текущем подключении?